The Ch(e)at Code

Agent Patterns for Copilot Chat

B A START

ISSUE #010 • JUNE 2, 2026 • 🧠 CONCEPTUAL PATTERN

Your customer built an HR agent. It works. Now they want an IT support agent, a finance agent, and a facilities agent. Great — until an employee asks the HR agent about laptop provisioning for a new hire. The HR agent doesn't know. The IT agent does. But the user has to start over, re-explain the context, and hope they picked the right bot this time. The moment you have two agents, you have a handoff problem.

This issue's pattern: Multi-Agent Handoff — a framework for when to split agents, how to transfer conversations between them, and where to draw the trust boundaries that keep the system coherent. If you've read Issue #005 (Prompt Chaining) and Issue #008 (Scoped Knowledge), this is what happens when those patterns need to talk to each other.

🧩 PATTERN BREAKDOWN

Multi-Agent Handoff

Three handoff models + trust boundaries + shared context architecture

Why this matters: This is the #1 blocker to multi-agent deployment at enterprise scale. Customers start with one agent, prove value, then want five more. The technical challenge isn't building the second agent — it's connecting them. CSAs who can solve the handoff problem own the expansion conversation.

The Cognitive Load Test: When should you split into multiple agents? Two signals: (1) agent scope exceeds ~3 knowledge domains, or (2) the system prompt exceeds ~2,000 tokens of instruction. But when domains overlap significantly or users can't predict which agent to ask, keep it as one agent with a **"receptionist" pattern** — a single front-door agent that routes to specialists behind the scenes.

```
THREE HANDOFF PATTERNS

Cold Transfer — restart with new agent, user repeats
context
  Simple ✓  |  Terrible UX ✗
Warm Transfer — Agent A summarizes, passes context to
Agent B
  Good UX ✓  |  Moderate complexity ◆
Hot Transfer — shared memory store, both agents
read/write
  Seamless UX ✓  |  High complexity ✗
```

The pattern has three design decisions:

1. **Cold transfer: simple but brutal.** The conversation ends with Agent A and restarts with Agent B. The user has to re-explain their problem. This is what most multi-bot deployments do today, and it's why users hate them. The only time cold transfer is acceptable: when the domains are so different that no context would help (e.g., handing off from an internal agent to an external vendor's support bot). In every other case, you need at least a warm transfer.

2. **Warm transfer: the sweet spot for most builds.** Before handing off, Agent A generates a conversation summary — typically via an Azure OpenAI call: *"Summarize this conversation in 3 sentences, including the user's original question and what's been discussed so far."* This summary is passed to Agent B as a context variable. Agent B's greeting topic reads it and picks up mid-stride. Good UX, moderate complexity, and the approach we'll build in Issue #011. This is what you should default to.

3. **Hot transfer: shared memory for advanced scenarios.** Both agents read and write to the same session state — a Dataverse table, Redis cache, or external orchestrator. The user sees one continuous conversation even though multiple agents are serving it. This is architecturally complex but necessary when agents collaborate within a single workflow (e.g., an approval chain that spans HR and Finance). Reserve this for scenarios where warm transfer's slight context gap is unacceptable.

Trust boundaries matter: Should Agent B trust Agent A's intent classification? What if Agent A misrouted — how does Agent B detect and recover? Does the user need to re-authenticate with Agent B? These aren't edge cases; they're the questions that determine whether your multi-agent system feels coherent or feels like being transferred between call centers. Build a misroute detection topic in every receiving agent —

if the user's first message after transfer doesn't match the expected domain, route them back.

WHERE TO STORE SHARED CONTEXT

```
Conversation variables — lightweight, ephemeral, lost on session end
  Best for: warm transfer summary passing
Dataverse session table — persistent, queryable, auditable
  Best for: analytics, audit trails, multi-step workflows
External orchestrator — Azure Bot Framework, API-based routing
  Best for: complex multi-agent topologies with 5+ agents
```

⚠ **Design warning:** Don't build a hot transfer when a warm transfer will do. Shared session state adds architectural complexity, Dataverse dependencies, and debugging difficulty. Start with warm transfer (Issue #011 shows you how), measure whether the context gap matters, and upgrade to hot transfer only if users report losing context mid-handoff. Premature shared state is the #1 over-engineering mistake in multi-agent builds.

🏆 WHERE THIS PATTERN LANDS

- **HR + IT shared services** — the canonical example. New hire onboarding touches both domains. An employee asks the HR agent about laptop provisioning — the agent should route to IT with full context, not say "contact IT support."
- **Customer service tiers** — general inquiry agent routes to billing, technical support, or account management based on intent. Warm transfer carries the customer's issue summary so they never repeat themselves.
- **Regulated industries (finance, healthcare)** — compliance requires that certain queries are handled only by agents with access to specific data. Handoff isn't a convenience — it's a policy enforcement mechanism.
- **Any enterprise with 2+ Copilot Studio agents** — this is the pattern that turns a collection of bots into a coherent agent ecosystem. Without it, every new agent is an island.

Positioning tip: Frame this as "agent ecosystem, not agent collection." The customer's first agent proved AI works. The second agent proves they need architecture. You're the CSA who turns a successful pilot into a scalable platform — and that conversation starts with handoff.

⚡ **QUICK TIPS**

✓ **The "receptionist" test.** Ask your customer: "Can your users predict which agent to talk to?" If the answer is no, don't deploy multiple user-facing agents — deploy one front-door agent that routes internally. The user never sees the handoff. This is cheaper to build and dramatically better UX.

✓ **Pair with Issue #005 (Prompt Chaining)** for the pattern that structures *within*-agent multi-step workflows, and **Issue #008** for scoped knowledge boundaries. Multi-Agent Handoff is the *between*-agent coordination layer.

✓ **Next week: we build it.** Issue #011 takes this conceptual framework and implements a two-agent warm handoff in Copilot Studio with Dataverse session state and handoff analytics. If you're reading this and thinking "show me the code," that's next Monday.

Got an agent or pattern to share?

Reply to this email and you might be featured in a future issue.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution